

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC THĂNG LONG**

**BÁO CÁO MÔN:
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT**

LẬP TRÌNH GAME SKY INVADERS

Giảng viên hướng dẫn: CTI062 Cao Đình Hoàng
Minh

Nhóm thực hiện: Nhóm 4

Sinh viên thực hiện:

A51294 – Đỗ Trung Hiếu

A51376 – Nguyễn Minh Tuấn

A51578 – Lưu Thị Tuyết

A51590 – Đỗ Đình Việt

HÀ NỘI – 2026

BẢNG PHÂN CÔNG NHIỆM VỤ NHÓM 4

Link github file: <https://github.com/trunghieutkdh2006-dotrunghieu/sky-invaders-game>

MSSV	Họ và tên	Nhiệm vụ phân công	Mức độ đóng góp
A51294	Đỗ Trung Hiếu	Trưởng nhóm. Thiết kế kiến trúc tổng thể, cài đặt Game Loop, Player class, thiết lập các kiểu bắn của enemy, xử lý sự kiện bàn phím, tích hợp SDL2.	25%
A51376	Nguyễn Minh Tuấn	Cài đặt Enemy class, thuật toán spawn ngẫu nhiên, thuật toán AABB Collision Detection, tích hợp hệ thống điểm số.	25%
A51578	Lưu Thị Tuyết	Cài đặt Bullet class, PowerUp system, Explosion effect, xử lý âm thanh	25%

MSSV	Họ và tên	Nhiệm vụ phân công	Mức độ đóng góp
		(SDL_mixer), thiết kế assets.	
A51590	Đỗ Đình Việt	Cài đặt Render module, GameState management, UI/HUD, viết báo cáo, kiểm thử và debug.	25%

Tất cả thành viên đều tham gia kiểm thử, họp nhóm và hoàn thiện báo cáo cuối. Mức độ đóng góp được đánh giá đồng đều 25% mỗi người do sự phối hợp chặt chẽ xuyên suốt quá trình phát triển.

CHƯƠNG 1. TỔNG QUAN VÀ PHÂN TÍCH YÊU CẦU

1.1. Giới thiệu đề tài

Trong những năm gần đây, ngành công nghiệp game phát triển mạnh mẽ và trở thành một trong những lĩnh vực giải trí phổ biến nhất trên thế giới. Các trò chơi điện tử không chỉ mang tính giải trí mà còn giúp người chơi rèn luyện khả năng phản xạ, tư duy logic và kỹ năng xử lý tình huống.

Trong quá trình học môn Cấu trúc dữ liệu và giải thuật, việc xây dựng một trò chơi bằng ngôn ngữ C++ kết hợp thư viện SDL2 giúp sinh viên áp dụng được các kiến thức về:

- Lập trình hướng đối tượng (OOP)
- Cấu trúc dữ liệu (vector, enum, struct)
- Giải thuật (AABB Collision, Game Loop, Random Spawn)
- Xử lý đồ họa 2D và âm thanh đa phương tiện
- Quản lý bộ nhớ và tối ưu hiệu năng chương trình

Đề tài "Sky Invaders" được xây dựng dựa trên phong cách game bắn máy bay cổ điển. Người chơi điều khiển phi thuyền chiến đấu trong không gian để tiêu diệt kẻ địch, né tránh va chạm và đạt số điểm cao nhất.

1.2. Mục tiêu đề án

- Xây dựng game hoàn chỉnh bằng C++ và SDL2
- Áp dụng cấu trúc dữ liệu và giải thuật trong gameplay
- Xử lý đồ họa 2D và âm thanh đồng thời
- Tạo hệ thống enemy, bullet và powerup
- Quản lý game bằng lập trình hướng đối tượng
- Xây dựng game loop ổn định, đạt FPS mục tiêu

1.3. Gameplay và luật chơi

1.3.1. Gameplay

Người chơi điều khiển một phi thuyền không gian ở phía dưới màn hình. Kẻ địch liên tục xuất hiện từ phía trên và di chuyển xuống dưới. Người chơi cần bắn hạ kẻ địch và né tránh va chạm để tồn tại lâu nhất có thể.

Các phím điều khiển:

DI CHUYỂN

A / ←	→	DI CHUYỂN SANG TRÁI
D / →	→	DI CHUYỂN SANG PHẢI
W / ↑	→	DI CHUYỂN LÊN
S / ↓	→	DI CHUYỂN XUỐNG

TẤN CÔNG

SPACE	→	BẮN ĐẠN
-------	---	---------

1.3.2. Luật chơi

- Mỗi enemy bị tiêu diệt sẽ cộng điểm vào bảng điểm
- Người chơi có số mạng giới hạn
- Khi va chạm với enemy sẽ bị mất máu hoặc mất mạng
- PowerUp giúp tăng khả năng chiến đấu (hồi máu, tăng tốc bắn)
- Game kết thúc khi người chơi hết mạng

1.4. Công nghệ sử dụng

- Ngôn ngữ lập trình: C++
- Thư viện đồ họa: SDL2

- Xử lý hình ảnh: `SDL_image`
- Xử lý âm thanh: `SDL_mixer`
- Hiển thị chữ: `SDL_ttf`
- IDE: VS Code
- Hệ điều hành: macOS
- Build system: Makefile

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ GAME

2.1. Các đối tượng trong game

Game Sky Invaders bao gồm các đối tượng chính sau:

2.1.1. Player

Player là phi thuyền do người chơi điều khiển.

Chức năng chính: di chuyển trái/phải, bắn đạn, nhận PowerUp, hiển thị số mạng còn lại và xử lý va chạm với kẻ địch.

2.1.2. Enemy

Enemy là các máy bay địch tự động.

Chức năng: spawn ngẫu nhiên từ phía trên màn hình, di chuyển xuống dưới với tốc độ có thể tăng dần theo thời gian, bị tiêu diệt khi trúng đạn.

2.1.3. Bullet

Bullet là đạn bắn ra từ player.

Chức năng: di chuyển lên trên với tốc độ cố định, kiểm tra va chạm với enemy, tự xóa khi ra khỏi màn hình.

2.1.4. PowerUp

PowerUp là vật phẩm hỗ trợ người chơi (hình trái tim).

Chức năng: hồi máu, tăng tốc độ bắn, tăng sức mạnh tạm thời.

2.2. Cấu trúc dữ liệu sử dụng

Bảng tổng hợp các cấu trúc dữ liệu được áp dụng trong game:

Cấu trúc dữ liệu	Ứng dụng trong game	Lý do lựa chọn
<code>std::vector<Bullet></code>	Quản lý danh sách đạn bắn ra	Kích thước thay đổi linh hoạt, thêm/xóa $O(1)$ cuối
<code>std::vector<Enemy></code>	Quản lý danh sách kẻ địch	Duyệt nhanh, spawn/remove linh hoạt
<code>enum GameState</code>	Quản lý trạng thái MENU/PLAYING/GAMEOVER	Dễ đọc, kiểm soát flow game rõ ràng
<code>struct Position</code>	Lưu tọa độ (x, y) của các đối tượng	Tái sử dụng, code gọn và rõ ràng

2.2.1. Vector quản lý bullet và enemy

Game sử dụng `std::vector` để lưu danh sách bullet và enemy. Vector phù hợp vì số lượng đối tượng thay đổi liên tục trong gameplay.

```
std::vector<Bullet> bullets;
std::vector<Enemy> enemies;
```

Khi player bắn, bullet mới được `push_back` vào vector. Khi bullet ra khỏi màn hình hoặc trúng enemy, nó được xóa khỏi vector bằng `erase()`.

2.2.2. Enum quản lý trạng thái game

```
enum GameState { MENU, PLAYING, GAMEOVER };
```

Enum giúp quản lý luồng game rõ ràng, dễ mở rộng và tránh dùng magic number.

2.3. Giải thuật sử dụng trong gameplay

2.3.1. Game Loop

Game hoạt động theo vòng lặp chính (Game Loop) liên tục:

```
while (running) {  
    handleEvents();    // Nhận input từ bàn phím  
    update();           // Cập nhật trạng thái game  
    render();           // Vẽ lên màn hình  
}
```

Đây là giải thuật cốt lõi giúp game hoạt động theo thời gian thực với FPS ổn định.

2.3.2. Thuật toán kiểm tra va chạm AABB

Game sử dụng thuật toán Axis-Aligned Bounding Box (AABB) để kiểm tra va chạm giữa bullet-enemy và player-enemy:

```
if (bulletRect.x < enemyRect.x + enemyRect.w &&  
    bulletRect.x + bulletRect.w > enemyRect.x &&  
    bulletRect.y < enemyRect.y + enemyRect.h &&  
    bulletRect.y + bulletRect.h > enemyRect.y)  
{ /* Collision detected */ }
```

Ưu điểm: đơn giản, dễ cài đặt, hiệu năng tốt cho game 2D.

2.3.3. Thuật toán spawn enemy ngẫu nhiên

Enemy được sinh tại vị trí ngẫu nhiên theo chiều ngang màn hình bằng hàm rand():

```
enemy.x = rand() % SCREEN_WIDTH;
```

Cơ chế này tạo tính đa dạng cho gameplay, tránh lặp lại và tăng tính thử thách.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG VÀ TRIỂN KHAI

3.1. Cấu trúc project

Project được tổ chức theo hướng module hóa để dễ bảo trì và mở rộng:

```
Project/  
├─ assets/                (hình ảnh, âm thanh, font)  
├─ audio.cpp / .h        (quản lý âm thanh)  
├─ bullet.cpp / .h       (hệ thống đạn)  
├─ enemy.cpp / .h        (hệ thống enemy)  
├─ gamestate.cpp/.h      (quản lý trạng thái)  
├─ powerup.cpp / .h      (hệ thống PowerUp)  
├─ render.cpp / .h       (render đồ họa)  
├─ types.h               (struct Position, constants)  
├─ main.cpp              (entry point)  
└─ Makefile
```

3.2. Thiết kế class

3.2.1. Class Bullet

```
class Bullet {  
public:  
    int x, y, speed;  
    void update(); // bullet.y -= speed  
    void render();  
};
```

3.2.2. Class Enemy

```
class Enemy {
```

```
public:
    int x, y, speed;
    void update(); // enemy.y += speed
    void render();
};
```

3.2.3. Class PowerUp

```
class PowerUp {
public:
    int x, y, type;
    void update();
    void render();
};
```

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ KIỂM THỬ

4.1. Kết quả đạt được

Sau quá trình phát triển qua 5 sprint, nhóm đã xây dựng thành công game Sky Invaders với đầy đủ chức năng cơ bản:

- Điều khiển player mượt bằng bàn phím
- Hệ thống bắn đạn liên tục
- Enemy spawn ngẫu nhiên và di chuyển tự động
- Collision detection AABB chính xác
- Hệ thống PowerUp (tìm hồi máu)
- Hiệu ứng nổ (Explosion animation)
- Âm nhạc nền và sound effect
- Hiện thị điểm số và số mạng trên HUD
- Màn hình Game Over khi hết mạng
- FPS ổn định ~60 FPS

4.2. Bảng kết quả kiểm thử

Nhóm đã tiến hành kiểm thử thủ công các chức năng chính của game. Kết quả được tổng hợp trong bảng sau:

STT	Chức năng kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
1	Di chuyển player	Player di chuyển	Hoạt động đúng	✓ Đạt
2	Bắn đạn (SPACE)	Bullet xuất hiện và bay lên	Hoạt động đúng	✓ Đạt

STT	Chức năng kiểm thử	Kết quả mong đợi	Kết quả thực tế	Trạng thái
3	Va chạm bullet – enemy	Enemy bị xóa, cộng điểm	Hoạt động đúng	✓ Đạt
4	Va chạm player – enemy	Player mất máu/mạng	Hoạt động đúng	✓ Đạt
5	Spawn enemy ngẫu nhiên	Enemy xuất hiện vị trí ngẫu nhiên	Hoạt động đúng	✓ Đạt
6	Thu thập PowerUp	Hồi máu / tăng tốc bắn	Hoạt động đúng	✓ Đạt
7	Âm thanh nền (BGM)	Nhạc phát liên tục	Hoạt động đúng	✓ Đạt
8	Hiệu ứng nổ (explosion)	Animation nổ khi trúng đạn	Hoạt động đúng	✓ Đạt
9	Game Over khi hết mạng	Hiện thị màn hình Game Over	Hoạt động đúng	✓ Đạt
10	FPS ổn định	FPS ≥ 60 khi chơi bình thường	~60 FPS ổn định	✓ Đạt
11	Enemy tăng tốc theo thời gian	Độ khó tăng dần	Chưa tăng rõ ràng	⚠ Chưa hoàn chỉnh

4.3. Hạn chế và phân tích lỗi

Mặc dù game đã hoạt động ổn định nhưng vẫn còn một số hạn chế phát hiện qua quá trình kiểm thử:

4.3.1. Enemy chưa tăng tốc rõ ràng theo thời gian

Hiện tại tốc độ enemy gần như không thay đổi đáng kể theo thời gian chơi, khiến độ khó không tăng lên rõ rệt. Hướng sửa: thêm biến timer và tăng speed enemy sau mỗi một khoảng thời gian nhất định.

4.3.2. Giật nhẹ khi tải ảnh

Khi nhiều enemy và bullet xuất hiện đồng thời, có hiện tượng giật nhẹ do tải texture. Hướng sửa: preload toàn bộ texture khi khởi động game thay vì tải động, áp dụng texture caching.

4.3.3. AI enemy đơn giản

Enemy hiện tại chỉ di chuyển thẳng từ trên xuống, chưa có hành vi truy đuổi hay né đạn. Hướng sửa: bổ sung state machine cho enemy với các trạng thái: di chuyển bình thường, truy đuổi, bắn lại.

4.4. Hướng phát triển

Trong tương lai, game có thể được mở rộng với các tính năng:

- Boss battle cuối màn với nhiều pha tấn công
- Nhiều loại enemy với hành vi khác nhau
- Weapon upgrade system (đạn đôi, đạn chùm)
- Multiplayer cục bộ hoặc trực tuyến
- Online leaderboard lưu điểm cao nhất
- Hệ thống save/load game
- Particle effect nâng cao
- Nhiều màn chơi (map) với background khác nhau

CHƯƠNG 5. TRẢ LỜI CÂU HỎI TRÊN LỚP

Câu hỏi 1: Làm sao để di chuyển hướng máy bay ngược chiều kim đồng hồ?

Trong game Sky Invaders, các enemy di chuyển theo quỹ đạo tròn thông qua hai hàm `moveOrbit()` và `moveCirclePlayer()`. Cả hai hàm đều cập nhật góc xoay bằng cách trừ giá trị góc mỗi frame:

```
// Hàm moveOrbit() - enemy xoay quanh tâm cố định
void moveOrbit(Enemy &en, int W, int H)
{
    en.angle -= 0.03f; // Góc giảm dần → chuyển động
    ngược chiều kim đồng hồ

    en.centerX = std::clamp(en.centerX, pad, (float)W
- pad);
    en.centerY = std::clamp(en.centerY, pad, (float)H
- pad - 50);

    // Tính vị trí mới theo công thức đường tròn
    en.x = en.centerX + cos(en.angle) * en.radius;
    en.y = en.centerY + sin(en.angle) * en.radius;
}

// Hàm moveCirclePlayer() - enemy xoay quanh vị trí
player
void moveCirclePlayer(Enemy &en, float px, float py)
{
```

```
en.angle -= 0.04f; // Góc giảm dần → chuyển động ngược chiều kim đồng hồ
```

```
// Xoay quanh player với bán kính cố định 150px  
en.x = px + cos(en.angle) * 150;  
en.y = py + sin(en.angle) * 150;  
}
```

Giải thích: Trong hệ tọa độ màn hình 2D, trục Y hướng xuống dưới (ngược với toán học thông thường). Do đó khi giá trị góc angle giảm dần (`angle -= ...`), vật thể sẽ xoay theo chiều ngược kim đồng hồ. Để đổi sang chiều xuôi kim đồng hồ, chỉ cần đổi dấu trừ thành dấu cộng:

```
// Đổi thành xuôi chiều kim đồng hồ  
en.angle += 0.03f; // moveOrbit  
en.angle += 0.04f; // moveCirclePlayer
```

Ngoài ra có thể điều chỉnh tốc độ xoay bằng cách thay đổi hệ số (0.03f, 0.04f): giá trị càng lớn thì xoay càng nhanh, giá trị càng nhỏ thì xoay càng chậm.

Câu hỏi 2: Muốn tăng tốc độ bắn của máy bay thì cần làm gì?

Tốc độ bắn của enemy được kiểm soát bởi hai cơ chế chính trong code.

Cơ chế 1 – Biến `rate` và `shootCooldown` trong hàm `updateEnemies()`:

```
// Công thức tính xác suất bắn mỗi frame  
int rate = 220 - (level * 2);  
if (rate < 60) rate = 60; // Giới hạn tối thiểu 60  
frame
```

```

// Kiểm tra cooldown trước khi bắn
if (en.shootCooldown <= 0)
{
    if (rand() % rate == 0) // Xác suất bắn = 1/rate
    mỗi frame
    {
        float bdx = px - en.x;
        float bdy = py - en.y;
        float blen = sqrt(bdx * bdx + bdy * bdy);
        if (blen == 0) blen = 1;

        float bulletSpeed = 2.5f + (level * 0.03f);
        if (bulletSpeed > 4.5f) bulletSpeed = 4.5f;

        bullets.push_back({en.x, en.y,
                           (bdx / blen) *
bulletSpeed,
                           (bdy / blen) *
bulletSpeed,
                           true, 1});

        en.shootCooldown = 15; // Cooldown 15 frame
        trước khi bắn tiếp
    }
}
else
{
    en.shootCooldown--;
}

```

```
}
```

Để tăng tốc độ bắn, có thể sửa như sau:

```
// Cách 1: Giảm hằng số khởi đầu để bắn thường xuyên hơn
```

```
int rate = 120 - (level * 2); // Thay 220 → 120
```

```
// Cách 2: Tăng hệ số level để độ khó tăng nhanh hơn theo thời gian
```

```
int rate = 220 - (level * 5); // Thay level*2 → level*5
```

```
// Cách 3: Giảm cooldown để bắn liên tục hơn
```

```
en.shootCooldown = 5; // Thay 15 → 5
```

Cơ chế 2 – Hàm getDifficulty() kiểm soát fireRate và bulletSpeed:

```
Difficulty getDifficulty(int level)
```

```
{
```

```
    Difficulty d;
```

```
    // fireRate: số frame tối thiểu giữa các lần bắn
```

```
    // Level 1 → 192 frame, Level 10 → 120 frame,
```

```
Level 20 → 40 frame (tối thiểu)
```

```
    d.fireRate = std::max(40.0f, 200.0f - level * 8);
```

```
    // bulletSpeed: tốc độ bay của đạn enemy
```

```
    // Level 1 → 3.2, Level 10 → 5.0, Level 20 → 7.0
```

```
    d.bulletSpeed = 3.0f + level * 0.2f;
```

```
        return d;  
    }
```

Để tăng tốc độ bắn toàn diện hơn từ hàm này:

```
// Giảm fireRate nhanh hơn theo level (bắn nhanh hơn  
từ sớm)  
d.fireRate = std::max(20.0f, 200.0f - level * 12);  
  
// Tăng tốc độ đạn để khó né hơn  
d.bulletSpeed = 4.0f + level * 0.5f;
```

Tóm tắt: Để tăng tốc độ bắn của enemy, cần tác động vào ba điểm trong code: giảm biến rate, giảm shootCooldown trong hàm updateEnemies(), và điều chỉnh fireRate cùng bulletSpeed trong hàm getDifficulty(). Kết hợp cả ba sẽ cho hiệu quả tăng độ khó rõ rệt nhất.

CHƯƠNG 6. TÀI LIỆU THAM KHẢO

1. [1] SDL2 Official Documentation. <https://wiki.libsdl.org/SDL2>
2. [2] Lazy Foo' Productions – SDL Tutorial.
<https://lazyfoo.net/tutorials/SDL>
3. [3] Bjarne Stroustrup. The C++ Programming Language, 4th Edition. Addison-Wesley, 2013.
4. [4] Robert Nystrom. Game Programming Patterns. Genever Benning, 2014.
5. [5] Adam Drozdek. Data Structures and Algorithms in C++, 4th Edition. Cengage Learning, 2012.
6. [6] Tài liệu môn Cấu trúc dữ liệu và giải thuật – Đại học Thăng Long, 2026.
7. [7] SDL2 Mixer Documentation. https://wiki.libsdl.org/SDL2_mixer
8. [8] SDL2 Image Documentation. https://wiki.libsdl.org/SDL2_image